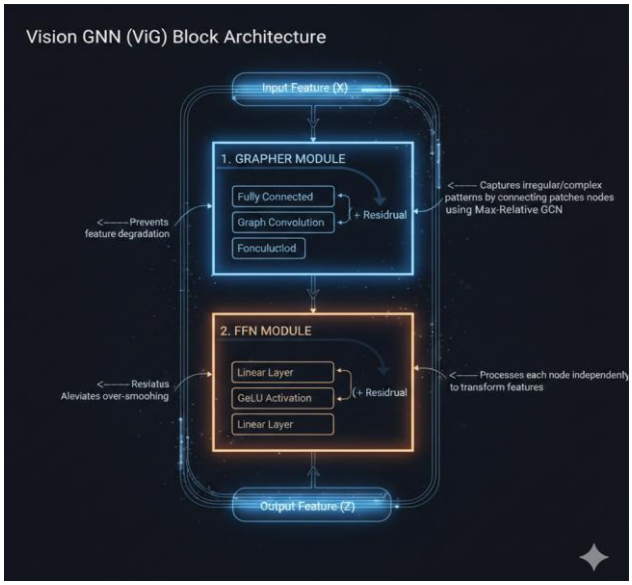
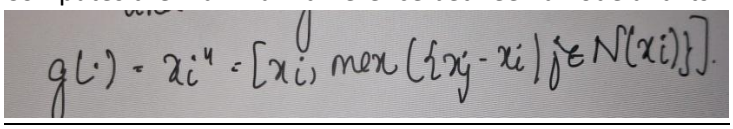
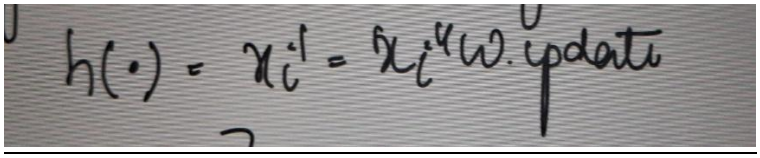
Graph Based Spreading Activations for Efficient Vision Models

VISION-GNN: paper (https://arxiv.org/abs/2206.00272)

- **Transforms the Standard image to Graph Representation**
  - Node construction: (H X W X 3) -> node patches -> each patch is a feature vector, which becomes a node in the graph
  - Edge Construction: For each node, the model identifies the nearest Neighbours based on feature similarity and adds directed edges from the Node to the neighbouring graphs
  - Returns a G(V,E) into a message passing framework
- **ViG Block**
  - The basic building block of the network, contains 2 main modules that extracts features while maintaining the diversity

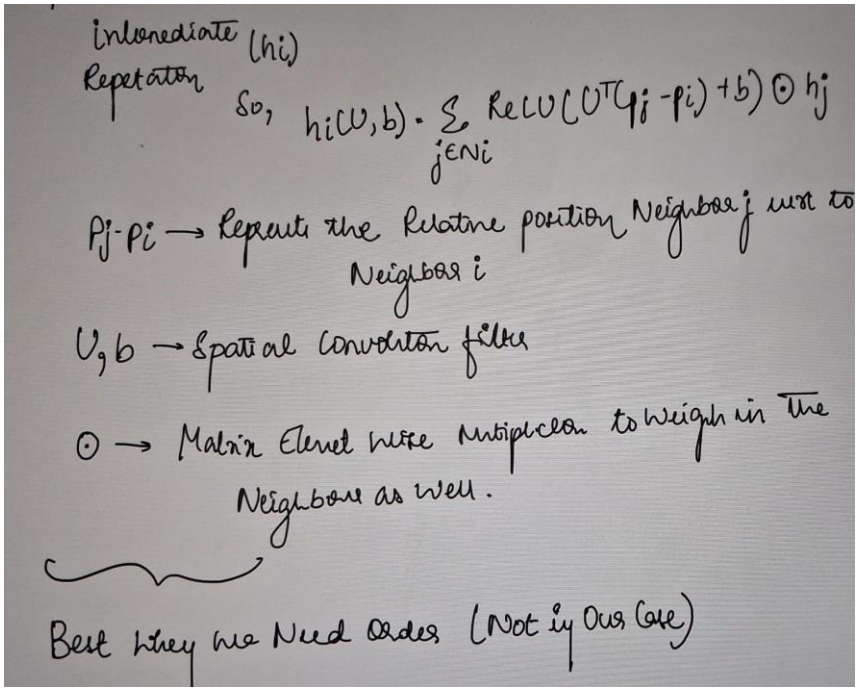


1. Grapher Module:
   - Performs Graph Convolutions to aggregate the information from the neighbouring nodes.
   - Adds Linear and residual Layers to not prevent degradation
2. FFN Module:
   - Helps Not smoothen on deep layer >3 layers like GCN does

- **"Max Relative Graph Convolutions"**
  - Aggregation:
    - Computes the maximum difference between a node and its neighbours
    - 
  - **Update:**
    - 
    - Merges with Features using learnable weights
    - Uses multi head update operations same as the Multi head attention allows to process multiple representation in the subspace
    - They have static layer method as well act like a ResNets by understanding Hierarchal shapes and then concentrating on local features slowly

- Note: Remove GCN and try to adapt ViG block from the VISIONGNN paper and see how works (Remarks: Aarya)

SPATIAL GRAPH CONVOLUTIONAL NETWORKS paper(https://arxiv.org/pdf/1909.05310):

- **Spatial Awareness in Graphs:**
  - Primary motivation is that the standard GCN does not keep the spatial order of the nodes, so treated as random order even though order has sense (ex: Molecular detection etc)
    - **Integrates Coordinates:**
      - Each nodes have a normal feature and then a spatial coordinate
    - **Geometric aggregation:**
      - Uses a relative weight by using a Anchor central Node and uses the feature vectors of those during aggregation
    - **Data Augmentation:**
      - Since it has a position encoded it allows augmentation through rotation and translations
  - **SGCM Mechanism:**
    - 

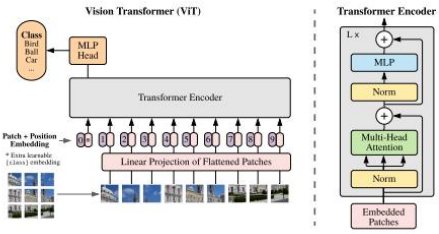
DYNAMIC-VIT: paper(https://arxiv.org/abs/2106.02034)

- **Attention is sparse:**
  - Self-attention is nothing but how much each token cares about the other tokens, researchers observed that many of the weights were 0
  - Through that they understood that only a subset of tokens is needed
  - They do not drop them at once (Static) but layer by layer based on Uncertainty of the Model, they keep ore tokens when the Uncertainty is high and keep lower tokens when there is lower uncertainty
- ***Prediction Modules:***
  - These are lightweight prediction modules that are between transformer blocks
  - Function: They give out importance scores per feature and use that to score each token

- Use binary masks 1(keep) else 0(discard token), using Gimbel Softmax
- Uses local context (MLP) and global context (CLS)

- ***ATTENTION MASKING***
  - Does not just prune the token directly they usually masks in the weight matrix
  - IF the Importance score we mask some j as 0 then the I -> j becomes 0 in the weight matrix or in the attention matrix hence not used during training

VIT: paper (https://arxiv.org/abs/2010.11929):



- *Transformers take 2D images into a feature vector (HXWXC)*
- *USES P^2 to get the resolution of the patch*
- *N = HW/P^2 is the length sequence that is passed into the transformer as input*
- *The sequence length remains the same throughout the different levels, only D gets flattened hence attention is present to every part of the image throughout the layers*
- *Uses Patch embeddings to understand positional Information*
- *"In the paper they have mentioned that 2D positional encoding made the most sense than using 2D positional Encoding"*
- ***Positional Embedding**: Transformers do not know the order of the input by default hence they use self-attention to trat inputs as a set and not as a sequence*
- Inject positional information into the Token embeddings
- ***CLS:*** The CLS token is main part is to "How to pool information from all the tokens that are a singular representation (aka becomes a learned summary of the input a global understanding)